GTM Role Dashboards — MCP App Technical Design

Oct 12, 2026 · @Andy P

Summary

We will build one remote MCP server that serves role-specific GTM dashboards as MCP Apps, rendered inline in Claude for Marketing Ops, BDRs, AEs and other sales roles. The server reads from Databricks, not live from ZoomInfo, Marketo or Salesforce. Headless Claude Code jobs keep the data fresh and write AI-generated summaries that the dashboards display.

The design reuses the attribution audit's foundations: deduplicated people and accounts, the canonical campaign ID, the campaign crosswalk, the enumerated action vocabulary and the lead- and account-level next-best-action (NBA). The dashboards become the read surface for that work; Marketo and Salesforce remain where actions happen.

MCP Apps is the first official MCP extension, co-developed by Anthropic, OpenAI and open-source contributors. Tools declare `ui://` HTML templates that the host renders in a sandboxed iframe, with a text fallback when the host has no UI support (Thoughtworks Radar). The dashboards render in Claude web, desktop and mobile, not in the Claude Code terminal.

Goals, non-goals and personas

The goal is one trusted, role-aware view of GTM data inside the tool reps and marketers already use to think. It is not a BI replacement or a write path into the source systems.

Goals

- Give each GTM role a dashboard tuned to its daily decisions, opened by asking Claude.
- Use one data foundation: deduplicated records, canonical campaign IDs and one account truth.
- Show assembled context, such as account research and the NBA with its reason, beside the numbers.
- Enforce row-level access by the viewer's identity, never by what the model passes in.
- Ship one role end to end first, then extend.

Non-goals for v1

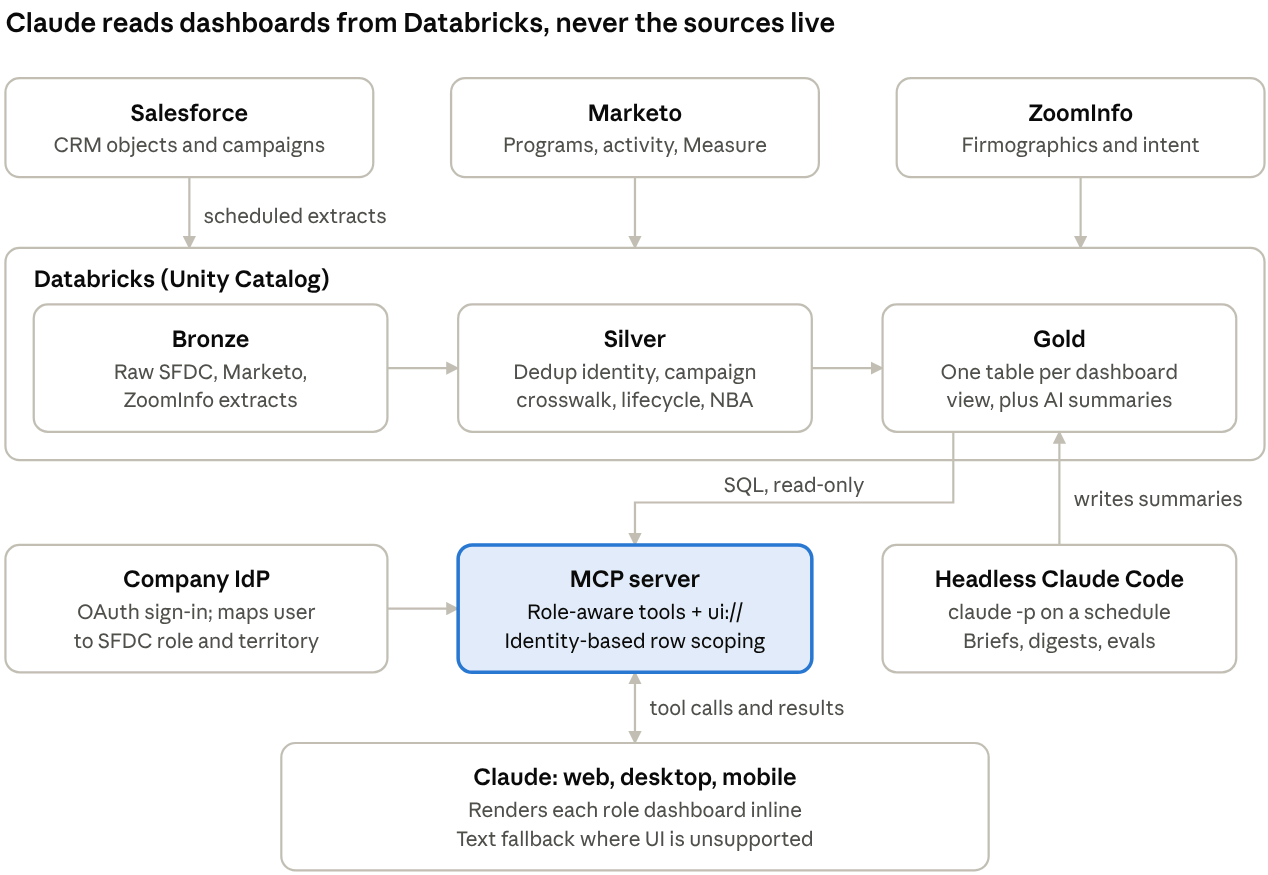
- Writing back to Salesforce or Marketo from the dashboard. Actions stay in those systems and their flows.
- Real-time data. Freshness is daily for most views and hourly where a role needs it.
- Replacing Marketo Measure for attribution or Revenue Cycle Modeler for lifecycle. The dashboards read their outputs.

Personas

Role	Daily question	Primary sources
Marketing Ops	Is the funnel healthy, and is attribution trustworthy?	Marketo, Salesforce campaigns, Marketo Measure
BDR / SDR	Who do I work today, and what do I say?	Marketo MQLs, ZoomInfo intent and firmographics, NBA
AE	Which deals and accounts need me this week?	Salesforce opportunities, account NBA, ZoomInfo
Sales / GTM leadership	Where is pipeline coming from, and is it converting?	All three, aggregated
Demand gen / ABM	Which programs and accounts are moving?	Marketo programs, account object, campaign crosswalk

Architecture overview

The system has four parts: scheduled ingestion into Databricks, a read-only MCP server, headless Claude Code jobs, and Claude as the rendering host. Only the ingestion jobs touch source-system APIs.



system architecture · 3 sources, 3 data layers, 1 MCP server

Read it top down. Sources land in bronze, the audit's identity and crosswalk logic builds silver, and gold holds one table per view. The MCP server reads gold under the viewer's identity, and Claude renders the result.

Data layer

Dashboards read only from Databricks gold tables, never from the source APIs at view time. Marketo enforces a daily API quota and short-window rate limits, and ZoomInfo bills by credit, so live fan-out per dashboard open would be slow, costly and fragile.

Ingestion (bronze)

Source	Method	Objects	Cadence
Salesforce	Existing ETL or Lakeflow connector	Lead, Contact, Account, Opportunity, Campaign, CampaignMember, Task, User, UserRole	Hourly
Marketo	Bulk Extract API (leads, activities, program members)	Leads, activities, programs, smart campaign membership, custom objects	Daily, with an intraday activity pull if needed
Marketo Measure	Its Salesforce objects or data warehouse export	Touchpoints, attribution touchpoints	Daily
ZoomInfo	Bulk enrich and intent export, scoped to known accounts and contacts	Firmographics, technographics, intent topics, contacts	Weekly enrich, daily intent

Conformed layer (silver) is where the audit work lives:

- **Identity:** deduplicated person and account keys, with survivorship rules and a log of where duplicates entered.
- **Campaign crosswalk:** the system-independent canonical campaign ID mapped to Marketo program IDs and Salesforce campaign IDs. Never join on campaign name.
- **Lifecycle:** stage history per person, reconciled between Marketo and Salesforce, with records that arrived lifecycle-blind flagged.
- **Action vocabulary:** the enumerated conversion actions that create touchpoints.

Serving layer (gold) has one table per dashboard view, already filtered to the grain the UI shows:

- `gold.mops_funnel_daily`, `gold.mops_attribution_by_campaign`, `gold.mops_data_quality`
- `gold.bdr_worklist` (person NBA, reason, ICP fit, intent, last touch)
- `gold.ae_account_brief` (account NBA, buying-group activity, open opportunities)
- `gold.ai_summaries` (text written by the headless jobs, keyed by role, owner and as-of time)

Every gold row carries `owner_user_id`, `territory_id` and `refreshed_at`. These fields drive row-level security and the freshness stamp on every dashboard.

Access path: the MCP server queries a Databricks SQL warehouse through the SQL Statement API. Unity Catalog row filters on `owner_user_id` and `territory_id` provide a second enforcement layer below the server.

MCP server and MCP Apps UI

The server is a TypeScript MCP server with the official MCP Apps SDK, served over Streamable HTTP. It exposes a small set of read-only tools, and each dashboard tool is bound to one `ui://` resource.

Stack

- `@modelcontextprotocol/sdk` for the server and `@modelcontextprotocol/ext-apps/server` to register app tools and UI resources (MCP Apps repo).
- `@modelcontextprotocol/ext-apps` in the view (`App` class, `PostMessageTransport`) for host communication.
- A Vite single-file build per dashboard. Each UI must be self-contained HTML, CSS and JS, because the sandbox does not load external URLs (skill reference).
- `@databricks/sql` or the SQL Statement REST API for queries.

Tools

Tool	UI resource	Returns	Notes
`open_dashboard`	none	The caller's role and which dashboard tool to call	Router; resolves the role from identity, not arguments
`mops_dashboard`	`ui://gtm/mops`	Funnel, attribution and data-quality payload	Filters: date range, segment, canonical campaign ID
`bdr_dashboard`	`ui://gtm/bdr`	Today's worklist with NBA, reason and context	Scoped to the caller's owned records
`ae_dashboard`	`ui://gtm/ae`	Account briefs and opportunity watchlist	Scoped to the caller's territory and accounts

Tool	UI resource	Returns	Notes
`leader_dashboard`	`ui://gtm/leader`	Aggregated pipeline sourcing and conversion	Aggregates only, no row detail
`get_account_brief`	none	Text brief for one account	Lets Claude reason about an account in chat
`explain_metric`	none	Definition, source table and as-of time	Keeps answers traceable

Contract for each dashboard tool

1. The tool returns `structuredContent` (the data the view renders) plus a short text `content` fallback. Hosts without MCP Apps support still get a usable answer.
2. The view receives the tool result, renders it, and can call server tools back through the host for drill-downs and filter changes. No new conversation turn is needed.
3. The view can push a message into the chat, for example "Draft an email to these five accounts". Claude then acts with the user in the loop.
4. Payloads stay small: pre-aggregated, capped at a few hundred rows, and paged through tool calls.

Repository layout

```
gtm-dashboards-mcp/
  server/
    index.ts           # transport, auth middleware, tool registration
    tools/             # one file per tool
    identity.ts        # token -> user -> role/territory
    warehouse.ts       # parameterized SQL, no string building
  views/
    mops/ bdr/ ae/ leader/   # one Vite app each -> single HTML file
    shared/            # charts, tables, theme tokens
  jobs/                # headless Claude Code prompts and runners
  CLAUDE.md            # conventions for Claude Code in this repo
```

Identity, role resolution and security

The signed-in user's identity decides both the role and the rows; the model never does. A role or owner passed as a tool argument is ignored for access decisions.

Flow

1. The user adds the server as a custom connector in Claude and signs in through OAuth against the company IdP (Okta or Entra).
2. The server validates the access token on every request and reads the user's email or IdP subject.
3. `identity.ts` maps the email to the Salesforce `User`, then `UserRole` and `Profile`, plus territory, from a cached `gold.user_access` table refreshed hourly.
4. A role-map config translates Salesforce roles to dashboard roles (MOps, BDR, AE, leader). An explicit override list covers edge cases.
5. Every query is parameterized with the resolved `user_id` and `territory_ids`. Unity Catalog row filters enforce the same scope again.

Credentials

- The server uses a Databricks service principal with read-only access to the gold schema. It never holds Salesforce, Marketo or ZoomInfo credentials.
- Source-system credentials live only in the ingestion jobs, in a secret scope.
- A user may request a broader view, for example a manager looking at their team. The server checks the role hierarchy before widening scope.

Data handling

- Tool results enter the Claude conversation. Exclude fields the org treats as restricted, such as personal phone numbers or anything under financial-services compliance review, from gold tables entirely.
- Log every tool call with user, tool, filters and row count. Do not log result data.
- Treat any text in a tool result as data. Free-text fields from CRM notes can contain prompt-injection attempts, so views escape them and AI summaries quote them rather than follow them.
- Get security and compliance review before broad rollout. Confirm the AI usage policy covers CRM data in Claude, and whether traffic must route through the company's Databricks-backed AI gateway.

Dashboard specs by role

Each dashboard answers one daily question, shows its freshness, and ends in an action the user takes in Salesforce or Marketo, or asks Claude to do.

Marketing Ops (build first)

- Funnel by lifecycle stage, with conversion and velocity between stages, filterable by segment and canonical campaign ID.
- Attribution by canonical campaign: touchpoints, sourced and influenced pipeline from Marketo Measure, and the share of opportunities with no attributable touch.
- Data quality panel: new duplicates by entry point (form, list import, campaign-member sync), records arriving lifecycle-blind, and key ICP fields unpopulated.
- MQL quality: MQLs by trigger (score threshold versus hand-raise), rep follow-up time and outcome against ICP fit.
- Drill-down: click a campaign to see member progression in both systems and any sync mismatches.

BDR / SDR

- Today's worklist: owned MQLs and high-intent contacts, ranked by NBA priority.
- Each row shows the NBA action from the enumerated vocabulary, the reason (last touch or trigger), ICP fit, ZoomInfo intent topics and when the NBA was computed.
- Account context drawer: the account-level NBA and buying-group activity. An account play can override a person-level NBA.
- Action buttons send a prompt to Claude, for example "Draft a first-touch email for this contact using the account brief".

AE

- Account watchlist: territory accounts with account NBA, tier, recent buying-group engagement and intent surges.
- Opportunity panel: open opportunities with stage age, last activity and marketing touches since the last stage change.
- Account brief: the AI summary from `gold.ai_summaries`, with its as-of time and sources.

Leadership

- Pipeline by source and canonical campaign, with conversion to closed-won.
- MQL-to-opportunity conversion by team and segment.

- Aggregates only: no person-level rows.

Shared UI rules

- Every view shows `refreshed_at` and the source tables used, available through `explain_metric`.
- One theme with light and dark support, and phone-width layouts, because many reps will open these on mobile.
- Empty and error states say what is missing and why, for example "No MQLs assigned to you in the last 7 days".

Headless Claude Code jobs

Claude Code runs non-interactively (`claude -p`) on a schedule to do the work that needs judgment: writing briefs, checking data and testing the server. Its outputs land in Databricks; dashboards only read them.

Job	Cadence	Reads	Writes
Account briefs	Daily, before 7am PT	`gold.ae_account_brief`, ZoomInfo intent, recent activity	`gold.ai_summaries` (role = AE)
BDR worklist notes	Daily	`gold.bdr_worklist`	`gold.ai_summaries` (role = BDR), one line per top record
Data quality digest	Daily	`gold.mops_data_quality`	`gold.ai_summaries` (role = MOps), anomalies with counts
Server evals	On every pull request and nightly	MCP server in a test environment	Pass or fail report in CI

How a job runs

- A CI runner, or a Databricks job calling a container, executes `claude -p "<prompt file>" --output-format json`, with tool permissions limited to what the job needs.
- The job reads through a read-only Databricks MCP server or SQL tool, and writes through a single `write_summary` tool that validates schema, role and owner.

- The prompt files live in `jobs/` and are versioned with the server, so changes are reviewed like code.
- Route model calls through the company AI gateway if policy requires it, for cost tracking and the GTM AI spend reporting.

Evals

- For each role, a fixture user calls its dashboard tool. Assert that the result shape is valid, that rows are scoped to that user only, and that the text fallback is present.
- A negative test: a BDR fixture passes another owner's ID as an argument and must get only its own rows.
- A render test loads each view's HTML in a headless browser with a fixture payload and checks for console errors.

Deployment, distribution and operations

Deploy the server as a stateless container behind the company's auth boundary. Then add its URL to Claude as an org custom connector so every licensed user can enable it.

Hosting options

Option	Pros	Cons
Databricks Apps	Close to the data; service principal and Unity Catalog built in	Check that it supports public HTTPS ingress and OAuth for an external MCP client
Cloud Run, ECS or the internal platform	Standard ops, easy autoscaling, familiar to platform teams	Needs its own networking path to the SQL warehouse

Distribution

- A Claude org admin adds the server URL as a custom connector. Users enable it per chat and sign in once.
- Each tool description says what it is for and who it is for, so Claude picks the right tool without users needing to know tool names.
- Publish a short how-to: "Ask Claude: show my dashboard" plus two or three example prompts per role.

Development loop in Claude Code

1. Install the MCP Apps plugin: `/plugin install mcp-apps@modelcontextprotocol-ext-apps`. It scaffolds new apps and adds UI to existing servers.
2. Build against fixture data first, then point to a dev SQL warehouse.
3. Test rendering in Claude Desktop pointed at the local server; MCP Inspector checks tool shapes but does not fully render apps.
4. CI runs the evals on every pull request; merging to main deploys.

Operations

- Health: track tool latency and error rate per tool, and set an alert when gold tables miss their freshness target.
- Cost: warehouse query cost per dashboard open, and model cost of the headless jobs. Both feed the GTM AI spend reporting.
- Adoption: weekly active users by role and tool calls per user. These are the main success metrics for the pilot.

Rollout plan

Ship Marketing Ops first, because that data is closest to the audit and the team can judge correctness. Each phase has a gate before the next starts.

1. **Foundation:** gold tables for MOps, the `user_access` mapping, a server skeleton with OAuth and `open_dashboard`, and evals in CI.
   - Gate: MOps numbers reconcile with existing Salesforce and Marketo Measure reports within an agreed tolerance.
2. **MOps pilot:** `mops_dashboard` live for the MOps team only, with the data quality digest job.
   - Gate: the team uses it weekly and has no open correctness issues.
3. **BDR pilot:** `bdr_dashboard` with person-level NBA and ZoomInfo context, for a small group of BDRs and their manager.
   - Gate: the row-scoping negative tests pass, and BDRs report the worklist matches what they would choose.
4. **AE and account briefs:** `ae_dashboard`, account-level NBA and the daily brief job.
   - Gate: security and compliance sign-off for broad rollout.
5. **Org rollout and leadership view:** the org-wide connector, `leader_dashboard`, the how-to and adoption tracking.

Risks and open questions

The largest risk is trust: if the first dashboard shows numbers that disagree with existing reports, adoption stalls. That is why MOps goes first and reconciliation is the first gate.

Risks

Risk	Impact	Mitigation
Duplicates and unresolved lifecycle data make numbers look wrong	Users lose trust	Ship after the audit's identity and lifecycle fixes; show data-quality flags in the view
Data scope leaks across owners	Compliance incident	Identity-based scoping, Unity Catalog row filters, negative tests in CI
Marketo API quota exhausted by extracts	Marketo flows and sync degrade	Bulk extract only, a scheduled window, and a quota monitor
ZoomInfo credit burn	Unplanned spend	Enrich known records only, cache results, weekly cadence
Prompt injection through CRM free text	Wrong or unsafe agent actions	Escape text in views; treat tool output as data; no write tools in v1
Host support for MCP Apps differs by surface	Some users see only text	Every tool returns a useful text fallback

Open questions

- [ ] Does policy require Claude traffic for this use case to route through the Databricks-backed AI gateway?
- [ ] Which host: Databricks Apps or the internal platform? Who owns on-call?
- [ ] Is ZoomInfo data licensed for display to all roles, or only some?
- [ ] What is the source of truth for territory: Salesforce territory management or a separate table?
- [ ] What reconciliation tolerance does the MOps gate use?
- [ ] When does v2 add write actions, such as creating a Salesforce task from the BDR view, and under what approval model?